Resposta de Lista de Exercícios

Q1

QUESTION	(100 %)

Uma subsequência monotônica não-decrescente em x é uma sequências de elementos em x tal que $1 \leq i_1 \leq i_2 \leq \cdots i_k \leq d$ e $x_{i_1} \leq x_{i_2} \cdots x_{i_k}$. Encontre o comprimento da maior subsequência monotônica não-decrescente. Por exemplo, o comprimento da maior subsequência monotônica não-decrescente de $4, 6, 5, 9, 1$ é 3. *Justifique todas as suas opções e escolhas, assim como, deixe claro sua modelagem e os custos computacionais.*

- Comprimento da maior subsequência monotônica não decrescente (*Longest Increasing Subsequence* - LIS).
- **Método 1:**
  Modelar o problema como um **grafo direcionado** $G = (V, E)$, onde V contém os elementos da sequência X, e E contém uma aresta de X_i para X_j se $X_i < X_j$ e $i < j$. Aplicando um algoritmo de ordenação topológica (ou caminho crítico), o número de iterações necessárias corresponde ao comprimento da LIS.
  - Custo $O(V + E)$.
- **Método 2:**
  Analisar cada elemento em relação aos anteriores. Se X_i for maior que algum elemento anterior X_j, então atualiza-se o valor de $LIS[i] = LIS[j] + 1$; caso contrário, mantém-se o valor inicial. O processo é repetido para todos os elementos da sequência, partindo de um vetor inicializado com 1 (ou $-\infty$, dependendo da modelagem).
  - Custo $O(n^2)$.

Q2

QUESTION	(100 %)

Uma subsequência contínua de uma lista L é um subsequência de elementos consecutivos de L. Por exemplo, se L é $5, 15, -30, 10, -5, 40, 10$ então $15, -30, 10$ formam uma subsequência contínua, no entanto $5, 15, 40$ não formam uma subsequência contínua. Projete um algoritmo com custo, no máximo, $O(n)$ para encontrar a subsequência contínua de soma máxima. No exemplo acima a soma é 55. *Justifique todas as suas opções e escolhas, assim como, deixe claro sua modelagem e os custos computacionais.*

- Subsequência contínua de soma máxima (*Maximum Subarray Problem*).
- A cada iteração, mantém-se a soma parcial atual (`soma`) e o melhor resultado encontrado até o momento (`max_soma`). Se a soma parcial se tornar negativa, ela é reiniciada para 0,

pois uma soma negativa não pode contribuir para uma subsequência de soma máxima futura. Assim, percorrendo o vetor apenas uma vez, atualizando localmente as variáveis.

```
int max_soma(int n, int* L) // n = len(L)
{
  int max_soma = INT_MIN;
  int soma = 0;
  for(int i = 0; i < n; i++)
  {
      soma += L[i];
      if(soma > max_soma) max_soma = soma;
      if(soma < 0) soma = 0;
  }
  return max_soma;
}
```

- Custo $O(n)$.

Q3

QUESTION	(100 %)

Dado duas strings $x = x_1x_2\cdots x_n$ e $y = y_1y_2\cdots y_m$. Desejamos encontrar o comprimento da maior subsequência comum com custo computacional $O(mn)$ de forma que $x_{i_1}x_{i_2}\cdots x_{i_k} = y_{j_1}y_{j_2}\cdots y_{j_k}$, em que $i_1 < i_2 < \cdots < i_k$ e $j_1 < j_2 < \cdots < j_k$. *Justifique todas as suas opções e escolhas, assim como, deixe claro sua modelagem e os custos computacionais.*

- Comprimento da maior subsequência comum (*Longest Commom Subsequence* - LCS).
- Relação de Recorrência (Modelagem):

$$L(i,j) = \begin{cases} 0 & \text{, se } i = 0 \vee j = 0 \\ 1 + L(i-1, j-1) & \text{, se } X_i = X_j \\ max\{L(i, j-1), L(i-1, j)\} & \text{, se } X_i \neq X_j \end{cases}$$

```
int lcs_iterative(int n, int m, char* x, char* y) // n = len(x), m = len(y)
{
    int** L = (int**) malloc((n+1) * sizeof(int)); // n+1 linhas
    for(int i = 0; i <= n; i++) {
        L[i] = (int*) calloc((m+1), sizeof(int)); // cada linha com m+1
colunas
    }

    for(int i = 1; i <= n; i++)
```

```
    {
        for(int j = 1; j <= m; j++)
        {
            if(x[i-1] == y[j-1]) {
                L[i][j] = 1 + L[i-1][j-1];
            } else {
                L[i][j] = MAX(L[i-1][j], L[i][j-1]);
            }
        }
    }

    return L[n][m];
}
```

```
define MAX(a, b) ((a) > (b) ? (a) : (b))

int lcs_recursive_memoization(char* x, char* y, int i, int j) // i e j são
posições
{
    if (i == 0 || j == 0)
        return 0;
    if (memo[i][j] != -1)
        return memo[i][j];
    if (x[i - 1] == y[j - 1])
        memo[i][j] = 1 + lcs_recursive_memoization(x, y, i - 1, j - 1);
    else
        memo[i][j] = MAX(
            lcs_recursive_memoization(x, y, i - 1, j),
            lcs_recursive_memoization(x, y, i, j - 1)
        );

    return memo[i][j];
}
```

- Custo $O(m \times n)$.

Q4

QUESTION	(100 %)

Uma subsequência é "palindrômica"se ela é a lida da mesma forma da esquerda para a direita e da direita para a esquerda. Por exemplo, a sequência $A, C, G, T, G, T, C, A, A, A, A, T, C, G$ possui muitas subsequências palindrômicas, dentre elas, A, C, G, C, A e A, A, A, A. No entanto, A, C, T não é palindrômica. Projete um algoritmo, que execute em $O(n^2)$, tenha como entrada uma sequência com n caracteres, e que retorne o tamanho da maior subsequência palindrômica. *Justifique todas as suas opções e escolhas, assim como, deixe claro sua modelagem e os custos computacionais.*

- Tamanho da maior subsequência palindrômica.
- Para este problema, pode-se utilizar a LCS (*Longest Commom Subsequence*) na qual recebe como parâmetro a sequência X e o inverso de X, isto é:

$$\text{Tamanho da Maior Subsequência Palindrômica} = |LCS(X, inv(X))|$$

- Custo $O(n^2)$.

Q5

QUESTION	(100 %)

Projete um algoritmo para encontrar o maior valor, que pode ser obtido por uma colocação adequada de parênteses na expressão $x_1/x_2/x_3/\cdots/x_{n-1}/x_n$, em que x_i é um número positivo. *Justifique todas as suas opções e escolhas, assim como, deixe claro sua modelagem e os custos computacionais.*

- Assim como na multiplicação de matrizes, o valor final depende de como se escolhe o ponto de corte `k` entre `i` e `j`.
- Defini-se duas tabelas:
  - $M[i, j]$: o **valor máximo** possível da subexpressão
  - $m[i, j]$: o **valor mínimo** possível da subexpressão

  Essa duplicidade é necessária porque a divisão não é associativa nem monotônica, e o máximo em uma parte pode produzir mínimo global, dependendo da forma da expressão.
- Modelagem
  - Para maximizar a divisão, queremos dividir um número grande (numerador) por um número pequeno (denominador).
  - Para minimizar, fazemos o oposto.

$$M[X_i, X_j] = max\{\frac{M[X_i, X_k]}{m[X_{k+1}, X_j]}\}, i \leq k \leq j$$

$$m[X_i, X_j] = \max\{\frac{m[X_i, X_k]}{m[X_{k+1}, X_j]}\}, i < k < j$$

- Custo $O(n^3)$.

Q6

QUESTION	(100 %)

Seja um string de inteiros arbitrários $Z = \langle z_1, z_2, \cdots, z_k \rangle$, e peso(Z) $= \sum z_i$. Dado duas strings $X = \langle x_1, x_2, \cdots, x_m \rangle$ e $Y = \langle y_1, y_2, \cdots, y_n \rangle$. Projete um algoritmo para encontrar a subsequencia de peso máximo Z entre X e Y. *Justifique todas as suas opções e escolhas, assim como, deixe claro sua modelagem e os custos computacionais.*

- Encontrar uma sequência $Z = \langle z_1, z_2, \ldots, z_k \rangle$ tal que:
  - (Z) é subsequência de (X) e subsequência de (Y),
  - O peso total é máximo possível.
- O problema da subsequência comum de peso máximo é uma generalização direta do problema da LCS. A única diferença essencial está no termo somado na recorrência.

$$L(i,j) = \begin{cases} 0 & \text{, se } i = 0 \vee j = 0 \\ x_i + L(i-1, j-1) & \text{, se } X_i = X_j \\ max\{L(i, j-1), L(i-1, j)\} & \text{, se } X_i \neq X_j \end{cases}$$

- Custo $O(m \times n)$

Q7

QUESTION	(100 %)

Sejam duas strings X e Y, uma string Z é uma super-string comum de X e Y, se X e Y são subsequências de Z. Projete uma solução, usando programação dinâmica, que dadas duas strings X e Y, retorne o comprimento da Menor Supersequência Comum (MSC) de X e Y. *Justifique todas as suas opções e escolhas, assim como, deixe claro sua modelagem e os custos computacionais.*

- Comprimento da Menor Supersequência Comum (MSC)
- **Método 1**:
  O tamanho da MSC pode ser encontrado somando o tamanho da sequência X com a sequência Y e subtrair da LCS de X e Y, isto é:

$$\text{Tamanho da Menor Supersequência Comum} = |X| + |Y| - |LCS(X,Y))|$$

- **Método 2**:

$$MSC(i,j) = \begin{cases} 0 & , \text{ se } \ i = 0 \wedge j = 0 \\ i & , \text{ se } \ j = 0 \\ j & , \text{ se } \ i = 0 \\ 1 + MSC(i-1, j-1) & , \text{ se } \ X_i = X_j \\ 1 + min\{MSC(i, j-1), MSC(i-1, j)\} & , \text{ caso contrário} \end{cases}$$

Se os caracteres são iguais, eles fazem parte da supersequência.
Caso contrário, precisamos **incluir um novo caractere** de uma das sequências e escolher o caminho mínimo

- Custo $O(|X| \times |Y|)$.

Q8

QUESTION	(100 %)

Maior subsequencia crescente. Seja um vetor A de inteiros distintos. Encontre o comprimento da maior subsequência crescente de elementos em A. Por exemplo, para a sequência $[1, 19, 5, 10, 2, 50, 23, 35]$, a maior subsequência é $[1, 5, 10, 23, 35]$ e possui tamanho 5. *Justifique todas as suas opções e escolhas, assim como, deixe claro sua modelagem e os custos computacionais.*

- Maior subsequência crescente.
- Semelhante a Q1, mas nesse caso não queremos o tamanho da LIS e sim a própria sequência.
- Analisar cada elemento em relação aos anteriores. Se X_i for maior que algum elemento anterior X_j, então atualiza-se o valor de $LIS[i] = LIS[j] + 1$; caso contrário, mantém-se o valor inicial. O processo é repetido para todos os elementos da sequência, partindo de um vetor inicializado com 1 (ou $-\infty$, dependendo da modelagem).
- Para também recuperar a subsequência propriamente dita, mantém-se um vetor auxiliar `prev[i]`, que armazena o índice do elemento anterior na subsequência ótima terminando em X_i. Ao final do preenchimento, identifica-se o índice `end` que contém o maior valor em `LIS`, e reconstrói-se a sequência percorrendo `prev[end]` recursivamente (ou iterativamente) até o início da subsequência.

```c
int lis_iterative_sequence(int n, int* x, int** seq_out, int* seq_len)
{
    if (n == 0) {
        *seq_out = NULL;
        *seq_len = 0;
        return 0;
    }

    int* dp = (int*) malloc(n * sizeof(int));
    int* prev = (int*) malloc(n * sizeof(int));
```

```c
    for (int i = 0; i < n; i++) {
        dp[i] = 1;
        prev[i] = -1;
    }

    int best = 1;
    int end = 0;

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (x[j] <= x[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
                prev[i] = j; // guarda o anterior
            }
        }
        if (dp[i] > best) {
            best = dp[i];
            end = i;
        }
    }

    // reconstrução da sequência
    *seq_len = best;
    *seq_out = (int*) malloc(best * sizeof(int));

    int idx = best - 1;
    for (int i = end; i != -1; i = prev[i]) {
        (*seq_out)[idx--] = x[i];
    }

    return best;
}
```

- Custo $O(n^2)$.

Vinícius Miranda de Araújo